

Label-Efficient Object Detection and Tracking in Military Drone Imagery: A Comparative Experimental Study of Supervised Detectors and Self-Supervised Backbones

CSE 445 (Computer Vision), Group 02

Supervisor: Dr. Mohammad Rifat Ahmmad Rashid

Md. Ariful Islam Opi – 2023-1-60-141

MD Shafin Ahamed Siam – 2023-1-60-007

Albin Israt Siddique – 2023-1-60-212

Nasrullah Kaisher Sijan – 2023-1-60-204

Department of Computer Science and Engineering, East West University, Dhaka, Bangladesh

Abstract—Object detection in military drone imagery is constrained by the high cost of expert annotation, motivating two complementary questions addressed in this report. Part A asks which supervised detector architecture is best suited to the KIIT-MiTA dataset (seven classes of top-down aerial military hardware): we train and evaluate YOLOv10-s, YOLOv12-s, YOLOv26-s, and RF-DETR-nano, finding RF-DETR-nano the most accurate overall (test mAP50-95 = 0.526) but selecting YOLOv26-s (mAP50-95 = 0.460, 82 FPS) as the backbone carried into Part B, since Part B’s self-supervised weight-surgery pipeline requires a YOLO-family convolutional backbone that RF-DETR’s transformer architecture cannot provide. Part B asks whether self-supervised learning (SSL) pretraining on an 80% unlabelled image pool can reduce the labelled-data budget needed to reach competitive detection performance: we pretrain SimCLR, BYOL, I-JEPA, and a DINOv3-guided distillation on the unlabelled pool, transfer each backbone into YOLOv26s via weight surgery, and fine-tune at a 20% labelled budget. SimCLR wins this comparison (mAP50-95 = 0.0642), narrowly ahead of BYOL (0.0631) and I-JEPA (0.0601), with the DINOv3-distilled backbone trailing substantially (0.0396) due to an unresolved domain gap between its web-scale teacher and our narrow aerial domain. Critically, no SSL method approaches the COCO-pretrained supervised baseline (0.2767) at the same label budget, and a five-point nested label-efficiency ablation (10–50% labels) shows this gap persisting rather than narrowing across the entire studied range, with break-even never reached. We deploy the winning SimCLR-initialised detector in a ByteTrack/BoT-SORT tracking pipeline over two video clips, evaluated with proxy metrics. We report these results, including the negative SSL finding, as evidence-based conclusions with a concrete practical recommendation: for this problem domain, additional annotation effort produces more downstream benefit than additional SSL pretraining engineering.

Index Terms—object detection, self-supervised learning, label efficiency, multi-object tracking, YOLO, RF-DETR, SimCLR, BYOL, I-JEPA, DINOv3, military imagery

I. INTRODUCTION

Detecting military hardware (tanks, artillery, radar installations, missile launchers, soldiers, and support vehicles) in top-down drone imagery is a problem for which annotated

data is scarce by construction: the imagery is often restricted, the objects are rare, and correct labelling demands familiarity with military hardware silhouettes seen from an aerial vantage point that ordinary crowd-annotation pipelines cannot supply. A detector that performs well with a small labelled budget is therefore of direct practical value in this domain, and this report is organised around two experimental parts that jointly investigate how to obtain one.

Part A asks a model-selection question: among modern supervised detector architectures, which is best suited to this dataset, under a fixed compute budget (Kaggle, dual T4 GPUs)? We train and evaluate four architectures, namely YOLOv10-s, YOLOv12-s, YOLOv26-s, and RF-DETR-nano, to establish both an accuracy ceiling and the specific backbone architecture that Part B builds on.

Part B asks a label-efficiency question: if a detector’s backbone is pretrained on unlabelled imagery via self-supervised learning (SSL) before any labels are shown, how much labelled data can be saved relative to training from scratch or from a generic supervised (COCO) initialisation? We pretrain four SSL methods spanning three distinct learning paradigms, namely SimCLR (contrastive), BYOL (self-distillation without negative pairs), I-JEPA (masked latent prediction), and a DINOv3-guided teacher-student distillation, on an 80% unlabelled pool, transfer each into the Part A-selected YOLO backbone, and fine-tune at progressively smaller labelled budgets.

Our contributions, stated as claims substantiated in Sections IV–VI, are:

- RF-DETR-nano achieves the highest overall accuracy among four supervised detector architectures (mAP50-95 = 0.526) but is architecturally incompatible with the SSL weight-surgery pipeline used in Part B, making YOLOv26-s (mAP50-95 = 0.460) the correct backbone choice for the label-efficiency study despite not being the single most accurate Part A model.

- SimCLR is the strongest SSL pretraining method for this dataset among the four tested (test mAP50-95 = 0.0642 at a 20% label budget), narrowly ahead of BYOL and I-JEPA.
- DINOv3-guided distillation, despite originating from a considerably more powerful foundation-model teacher, under-performs all three from-scratch SSL methods (mAP50-95 = 0.0396), a result we attribute to an unresolved domain gap between web-scale pretraining and our narrow aerial-imagery domain within a fixed 200-epoch distillation budget.
- No SSL-initialised detector approaches the COCO-pretrained supervised baseline at any label fraction from 10% to 50%, and this gap does not narrow as labelled data increases, the opposite of what standard label-efficiency theory predicts, indicating that small-scale, narrow-domain SSL pretraining does not substitute for large-scale supervised pretraining in this setting.
- Detector quality propagates directly into tracking quality: the SimCLR-initialised detector, while the best available SSL backbone, produces measurably weaker tracking continuity (proxy metrics, Section V) than a stronger detector would be expected to.

Figure 1 previews both headline comparisons side by side: Part A’s four detector architectures (Section IV) and Part B’s four SSL backbones against their baselines (Section V), on a common mAP50-95 scale, establishing the two central numbers this report explains.

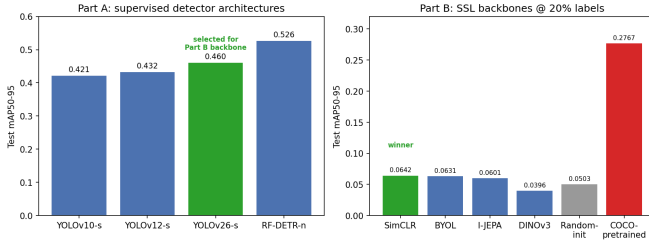


Fig. 1: Overview: Part A’s four supervised detector architectures (left) and Part B’s four SSL backbones plus baselines at the 20% labelled budget (right), both on test mAP50-95.

II. DATASET AND EXPERIMENTAL SETUP

A. Dataset

We use the KIIT-MiT A dataset [1], a 1,681-image, seven-class, 640×640 top-down aerial imagery collection of military hardware (*Artillery*, *Missile*, *Radar*, *M. Rocket Launcher*, *Soldier*, *Tank*, *Vehicle*), annotated in YOLO format. Table I summarises the per-class instance counts (4,586 total instances); the dataset exhibits a $4.10\times$ class-imbalance ratio between the majority class (*Vehicle*, 1,176 instances) and the minority class (*Radar*, 287 instances).

TABLE I: Per-class instance counts across the full KIIT-MiT A dataset.

Class	Instance count
Vehicle	1,176
Soldier	1,151
Tank	835
Missile	447
M. Rocket Launcher	386
Artillery	304
Radar	287
Total	4,586

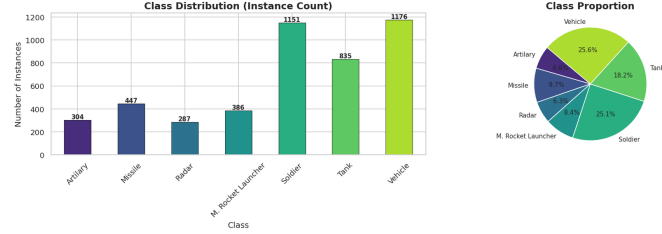


Fig. 2: Class distribution across KIIT-MiT A: instance counts (left) and proportional share (right). The $4.10\times$ imbalance ratio motivated the copy-paste augmentation described below.

B. Leakage-safe partitioning: the 10/10/80 protocol

A single, seeded (seed = 42) partition, generated once and reused unmodified by every notebook in this project (Part A and Part B alike), underlies every experiment:

- **Test (10.6%, 178 images):** reserved exclusively for final evaluation, in both Part A and Part B, never seen during any training or SSL pretraining.
- **Validation (9.8%, 164 images):** used for model selection/early stopping; excluded from the SSL pretraining pool.
- **SSL pool / training pool (79.7%, 1,339 images):** used directly as the supervised training set in Part A, and as the label-free pretraining pool in Part B. Because SSL pretraining never consults labels, no label information can leak through Part B’s pretraining stage by construction.
- **Labelled fine-tuning subsets (Part B only):** stratified subsamples of the SSL pool (with labels restored) at 10–50% of the pool, nested ($D_{0.1} \subset D_{0.2} \subset \dots \subset D_{0.5}$) and class-stratified, used for downstream detection fine-tuning.

Group-aware, class-stratified sampling (dominant per-image class as the stratification key) was used throughout, with near-duplicate images (identified via perceptual hashing, pHash threshold 10) kept within a single split as a group. Split disjointness was programmatically verified via filename-stem intersection checks in every notebook.

C. Preprocessing and augmentation

Whole-object copy-paste augmentation was applied to minority classes (*Artillery*, *Radar*, *M. Rocket Launcher*) on

training-side subsets only (never validation/test), targeting 60% of the majority class’s instance count.

TABLE II: Effect of copy-paste augmentation on the SSL pool’s class balance (train-split instance counts, before vs. after; source: dataset-eda notebook).

Class	Before	After
Artillery (minority)	240	610
Radar (minority)	225	772
M. Rocket Launcher (minority)	311	705
Missile	362	764
Tank	643	824
Soldier	907	937
Vehicle (majority)	881	1,033
Imbalance ratio (max:min)	$4.03\times$	$1.69\times$
Train images	1,339	2,377

Table II quantifies this effect directly: targeted synthetic instances were added for the three minority classes (Artillery, Radar, M. Rocket Launcher) toward 60% of the pre-augmentation majority-class count (target = 544 instances/class), while every other class also gained some collateral instances as a side effect of pasting synthetic objects onto donor images that already contained other classes’ boxes. The net effect is that the majority-to-minority instance-count ratio falls from $4.03\times$ before augmentation to $1.69\times$ after, at the cost of growing the on-disk training image count from 1,339 to 2,377 (validation and test splits are left untouched, so this asymmetry never affects evaluation).

An empirically-derived, dataset-wide decision, disabling rotation and vertical-flip augmentation (degrees=0.0, flipud=0.0), was applied uniformly across every training run in both Part A and Part B, because KIIT-MiTA’s consistent top-down aerial vantage point gives military hardware a canonical orientation that rotation destroys rather than usefully generalises over; we verified this empirically in Part A, where enabling rotation measurably reduced mAP.

D. Hardware and fair-comparison protocol

All experiments ran on Kaggle notebooks with two NVIDIA T4 GPUs. Part A’s four detectors were each trained under architecture-appropriate but otherwise consistent budgets (documented per-model in Section IV). Part B fixed every hyperparameter (epochs, batch size, image size, optimiser, augmentation policy) identically across all four SSL pretraining runs and all downstream fine-tuning runs (SSL-initialised and baseline alike), so that observed performance differences reflect the pretraining method rather than a confound in training budget.

E. Evaluation metrics

Detection is evaluated on the untouched test split using precision, recall, mAP50, and mAP50-95 (COCO-style, averaged over IoU thresholds 0.50–0.95), plus inference throughput (FPS) and parameter count for Part A’s architecture comparison. Tracking is evaluated with proxy metrics (Section V) in the absence of manually annotated ground truth: unique track-ID count versus a manually-established expected object count,

mean track length, fragmentation-event count, confidence distribution, and FPS.

III. METHODS

A. Part A: supervised detector pipeline

Four detector architectures were trained independently on the identical training/validation/test partition (Section II-B), each following a standard train–validate–test loop with a 1-epoch dry run first to verify the pipeline and estimate wall-clock time before committing to a full run:

- **YOLOv10-s** [4]: a single-stage, anchor-free CNN detector with consistent dual-label assignment for NMS-free inference.
- **YOLOv12-s** [5]: an attention-centric YOLO variant incorporating area-attention and residual efficient-layer-aggregation blocks.
- **YOLOv26-s** [3]: the most recent Ultralytics YOLO release at the time of this study, used both as a Part A candidate and, once selected, as the backbone architecture for all of Part B.
- **RF-DETR-nano** [6]: a DETR-family, transformer-based detector with a DINOv2-initialised backbone, trained with a considerably longer schedule (see Section IV) reflecting the slower convergence typical of transformer detectors relative to convolutional YOLO variants.

The best-performing architecture by test mAP50-95 *among YOLO-family candidates* was selected as the backbone for Part B (Section IV explains why RF-DETR, despite its higher raw accuracy, was not eligible for this role).

B. Part B: self-supervised pretraining methods

All four SSL methods pretrained a YOLOv26n backbone (yolo26n.yaml), or, for DINOv3, distilled into one, on the 1,339-image SSL pool, at 224×224 resolution, batch size 32/GPU across two T4s, for 200 epochs, with mixed precision and a 10-epoch warmup wherever the method supports one, so that any downstream performance difference reflects the SSL objective itself rather than a training-budget confound. Pretraining, weight surgery, and evaluation used the ssl-detection-lab (ssldet) library [2] provided by the course instructor.

TABLE III: SSL pretraining configurations.

Parameter	SimCLR	BYOL	I-JEPA	DINOv3
Teacher	None	EMA	EMA	ViT-B/16
Objective	NT-Xent	Bootstrap	Masked prediction	Distillation
Input size			224×224	
Batch size			32/GPU, 64 global	
Hardware			2 $\times$ NVIDIA T4	
Epochs			200	
Optimizer	AdamW	AdamW	AdamW	AdamW
Learning rate	3×10^{-4}	1×10^{-4}	3×10^{-4}	3×10^{-4}
Weight decay			1×10^{-4}	
Warm-up	10	10	10	Native
Precision	FP16	FP16	FP16	16-mixed
Workers			2	
Seed			42	

1) *SimCLR*: SimCLR [7] contrasts two augmented views of the same image against all other images in the batch via the NT-Xent loss,

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)}, \quad (1)$$

with temperature $\tau = 0.20$. Negative pairs are essential: without them, mapping every image to a constant embedding trivially minimises the positive-pair term, causing collapse. The projection head is discarded after pretraining; only the backbone encoder is transferred downstream (Section III-C), since the projection head specialises for instance discrimination and discards detail useful for detection.

2) *BYOL*: BYOL [8] removes the need for negative pairs via an asymmetric online/target network pair, the target updated only by an EMA of the online weights ($\tau_{\text{ema}} = 0.996 \rightarrow 1.0$). The online predictor learns to predict the EMA target’s projection of a second view, with stop-gradient on the target branch. Collapse is avoided through three mechanisms acting jointly: the stop-gradient prevents trivial copying; the predictor’s one-sided asymmetry destabilises the constant-output solution; and batch normalisation induces cross-example interaction within a batch. Our pretraining run reflects a stabilised configuration reached after early attempts at alternative momentum schedules exhibited collapse.

3) *I-JEPA*: I-JEPA [9] predicts masked target-block representations directly in embedding space rather than reconstructing pixels, avoiding the capacity waste of pixel-level reconstruction on high-frequency, semantically-irrelevant detail. Target blocks (4 blocks, scale $[0.10, 0.25]$, aspect ratio $[0.75, 1.50]$) are masked out of the context block so the prediction task cannot be solved trivially by copying. Because the masking-and-prediction task itself supplies an invariance signal, I-JEPA requires no hand-crafted augmentation policy, a property that sidesteps the dataset-specific augmentation failures we observed elsewhere (e.g., rotation).

4) *DINOv3-guided distillation*: Unlike the other three methods, our fourth branch distils knowledge from a *frozen*, already-pretrained DINOv3 ViT-B/16 teacher [10] (pretrained at web scale on LVD-1689M) into a randomly-initialised YOLOv26n student, both observing our own SSL pool, under a feature-matching distillation loss (AdamW, $\text{lr } 3 \times 10^{-4}$) for the same 200-epoch budget. This is domain-adaptive distillation rather than pretraining from scratch: the student is guided at every step by a teacher that already encodes rich, general-purpose visual features from a corpus far larger than our pool. This asymmetry matters for the fairness of our four-way comparison (Section V) and is revisited in the Discussion.

C. Weight surgery and downstream fine-tuning

Each SSL-pretrained encoder is transferred into a full YOLOv26n detector by loading its backbone weights into the corresponding detector backbone layers by name (*weight surgery*), verifying $\geq 95\%$ of backbone parameter tensors match by shape; the neck and head, with no SSL counterpart, retain standard random initialisation. Every fine-tuning

run (all four SSL-initialised detectors, plus random-init and COCO-pretrained baselines) then follows an identical two-stage protocol at the 20% labelled subset: (1) a 10-epoch head warm-up with the backbone frozen ($\text{lr } 1 \times 10^{-3}$), and (2) 140 epochs of full fine-tuning ($\text{lr } 3 \times 10^{-4}$, cosine decay, `close_mosaic=30`, patience 15), image size 640, batch 32, AdamW, rotation/flip disabled.

D. Tracking pipeline

The winning SSL-initialised detector (Section V) is deployed in a tracking-by-detection pipeline: the detector runs independently on every video frame, and ByteTrack [11] associates detections across frames into persistent identity tracks, using a Kalman filter to predict each track’s position before matching. ByteTrack’s key mechanism is two-stage association: high-confidence detections are matched first, and only then are remaining unmatched tracks matched against low-confidence detections that a naive tracker would discard, which is critical for surviving brief confidence drops under occlusion or motion blur without breaking the track. We additionally run BoT-SORT [12] (adding appearance re-identification and camera-motion compensation) on one clip for comparison. Because ground-truth track annotation was not performed, we evaluate with proxy metrics rather than MOTA/IDF1 (Section V).

IV. RESULTS: PART A

Table IV reports the consolidated test-set comparison across all four detector architectures.

TABLE IV: Part A: consolidated detector comparison, test split.

Model	mAP50	mAP50-95	Prec.	Rec.	F1	FPS	Params(M)
YOLOv10-s	0.646	0.421	0.673	0.600	0.635	84.7	7.22
YOLOv12-s	0.665	0.432	0.631	0.636	0.633	71.2	9.23
YOLOv26-s	0.648	0.460	0.722	0.624	0.669	82.1	9.47
RF-DETR-n	0.754	0.526	0.724	0.706	0.715	20.6	30.45

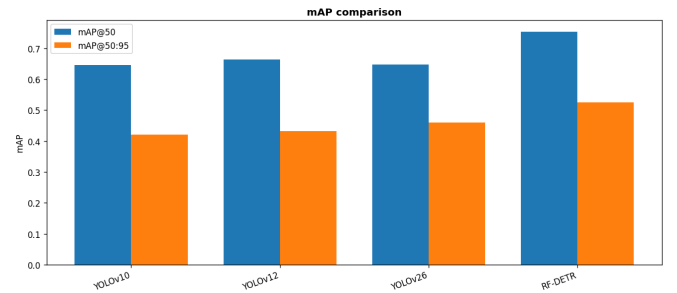


Fig. 3: Test-set mAP50 and mAP50-95 across all four Part A detector architectures.

A. Model-selection justification

RF-DETR-nano is the most accurate model overall (mAP50-95 = 0.526, +14% relative to the next-best YOLOv26-s), consistent with the general trend that DETR-family, transformer-based detectors with strong pretrained backbones (RF-DETR’s

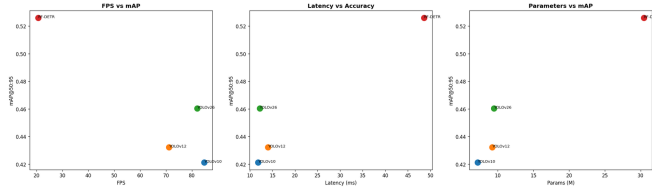


Fig. 4: Accuracy–speed–size trade-off: mAP50-95 vs. FPS, vs. latency, and vs. parameter count, for all four Part A detectors.

backbone is DINOv2-initialised) achieve higher raw accuracy at the cost of substantially higher latency (48.6 ms vs. 12.2 ms) and a $6\times$ longer training schedule (396 min vs. 86 min for YOLOv26-s), visible directly in Figure 4’s FPS-vs-mAP and latency-vs-accuracy panels, where RF-DETR is a clear Pareto outlier on accuracy but not on speed.

RF-DETR was nonetheless not eligible for the Part B backbone role. Part B’s central mechanism, weight surgery (Section III-C), requires loading an SSL-pretrained *convolutional* encoder’s weights directly into a YOLO-family backbone’s named layers. RF-DETR’s transformer backbone has no structural correspondence to a YOLO CNN backbone, so none of our four SSL pretraining methods (three of which train a YOLOv26n encoder from scratch, Section III-B) could be transplanted into it without an entirely different integration mechanism outside this project’s scope. Among the three YOLO-family candidates, **YOLOv26-s is the strongest** (mAP50-95 = 0.460, highest precision at 0.722, competitive FPS at 82.1), and was therefore selected as the Part B backbone: the best model *eligible* for the label-efficiency study, not the best model overall.

B. Qualitative results and robustness

Figure 7 shows representative YOLOv26-s test-set predictions against ground truth, and Figure 5 shows YOLOv26-s’s robustness under five image-corruption stress tests (Gaussian noise, Gaussian blur, brightness reduction, contrast shift, JPEG compression) at three severities each, run on a 120-image robustness subset. YOLOv26-s retains 82–97% of its clean mAP50 under mild-to-moderate corruption severity, with Gaussian noise and JPEG compression at the highest severity level producing the largest degradation (both to 82% of clean mAP50), indicating the detector is reasonably but not perfectly robust to common drone-imagery degradations (motion blur from platform vibration, compression artefacts from onboard video links).

V. RESULTS: PART B

A. SSL pretraining diagnostics

Figures 8–11 show pretraining loss (and, where applicable, EMA momentum) curves for all four SSL methods over the full 200-epoch budget.

We additionally verified SimCLR’s representation quality via t-SNE projection of validation-set embeddings (Figure 12), which shows visibly separable clusters by dominant object

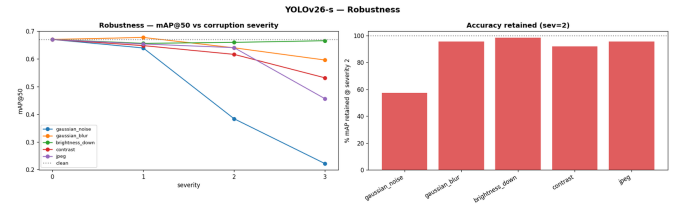


Fig. 5: YOLOv26-s robustness stress test: mAP50 retention under five corruption types at three severities each, relative to clean mAP50 on a 120-image subset.

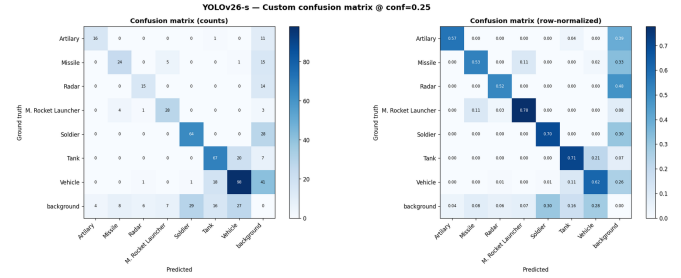


Fig. 6: YOLOv26-s confusion matrix on the test split (evaluation confidence 0.25).

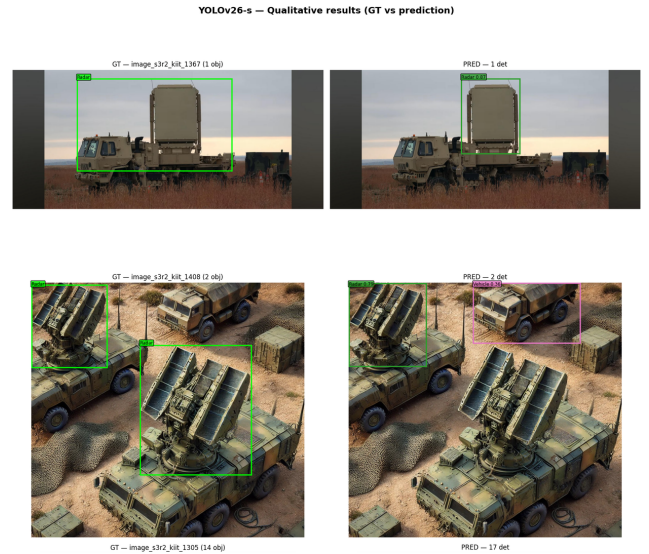


Fig. 7: Representative YOLOv26-s qualitative detections on held-out test images, spanning multiple classes and scene conditions.

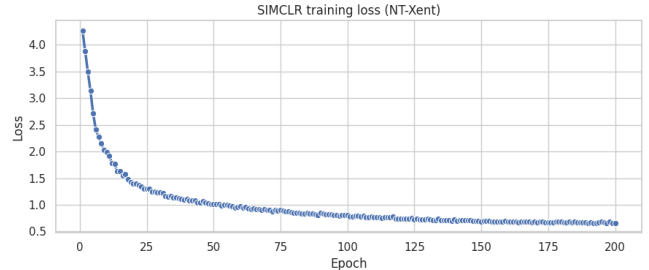


Fig. 8: SimCLR (NT-Xent) pretraining loss over 200 epochs.

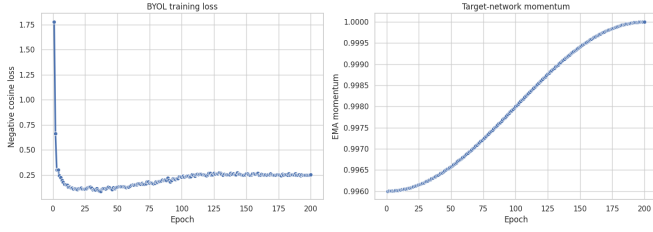


Fig. 9: BYOL training loss and EMA target-network momentum over 200 epochs.

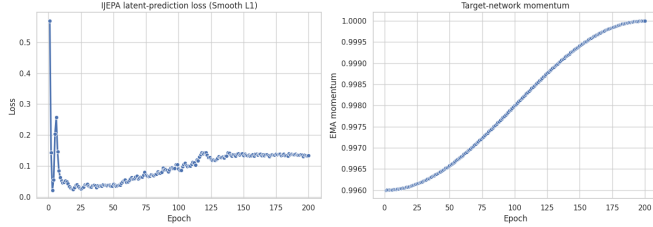


Fig. 10: I-JEPA representation-prediction loss and EMA target-network momentum over 200 epochs.

class despite the encoder never observing labels during pre-training.

1) *Shortcut-learning and representation-collapse checks:* Beyond the headline loss curves, every pretraining notebook ran two additional diagnostic checks on the main (200-epoch) run: a *shortcut-learning check*, testing whether the encoder had learned to trivially separate raw images from copy-paste-augmented synthetic ones rather than learning object semantics, and a *quantitative collapse diagnostic*, testing for representational collapse via the effective rank and pairwise-cosine-similarity statistics of validation embeddings (a fully collapsed encoder would show near-1.0 mean pairwise similarity and a near-zero effective rank). For SimCLR, raw and copy-paste-synthetic train-pool embeddings were visibly interleaved in t-SNE space rather than forming two separate clusters, indicating no shortcut-learning of the paste artefact.

Table V reports the quantitative collapse diagnostics for all four methods’ main runs. None show a strong collapse signature by this test, though I-JEPA’s markedly lower effective rank (26.3, versus 71–95 for the other three) indicates its validation

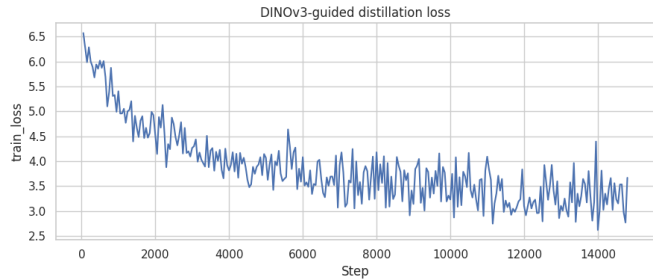


Fig. 11: DINOv3-to-YOLOv26n distillation loss over 200 epochs.

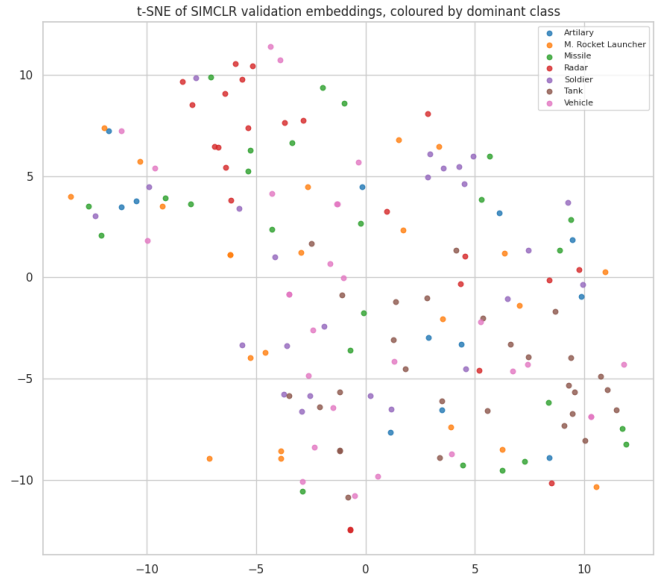


Fig. 12: t-SNE projection of validation embeddings from the SimCLR-pretrained encoder, coloured by dominant object class.

embeddings occupy a narrower effective subspace, worth flagging as a secondary, less severe form of representational under-utilisation even where outright collapse is absent.

TABLE V: Quantitative collapse diagnostics, validation-set embeddings, main (200-epoch) runs.

Method	Mean pairwise sim.	Std.	Eff. rank
SimCLR	0.148	0.215	79.5
BYOL	0.287	0.251	71.5
I-JEPA	0.306	0.408	26.3
DINOv3	0.420	0.135	94.8

2) *Per-method hyperparameter ablation exercises:* Each pretraining notebook also ran one method-specific hyperparameter ablation: a shorter (50-epoch) secondary run with one configuration change, to probe that method’s sensitivity to a key hyperparameter. Table VI summarises all four. The most quantitatively informative is BYOL’s: lowering the EMA target momentum from 0.996 (main) to 0.99 (exercise) raised mean pairwise similarity from 0.287 to 0.330 and *lowered* effective rank from 71.5 to 35.8, consistent with weaker online-target asymmetry increasing collapse risk, though the accompanying epoch-count reduction (50 vs. 200) is a confound. DINOv3’s exercise is also notable: initialising the student from COCO-pretrained weights before distillation *increased* mean pairwise similarity from 0.42 to 0.74, the opposite of what a “better-initialised” student might suggest, indicating its features were pulled toward the teacher’s generic representation on top of an already-generic initialisation, compounding rather than counteracting the domain-gap problem (Section VI).

TABLE VI: Per-method hyperparameter ablation exercises (50-epoch secondary runs).

Method	Config. changed	Observation
SimCLR	$\tau : 0.20 \rightarrow 0.10$	Sharper softmax; loss converges to a lower value
BYOL	momentum: 0.996 $\rightarrow$ 0.99	Higher similarity, lower eff. rank (Table V)
I-JEPA	4 $\rightarrow$ 8 target blocks, smaller scale	Loss spikes more, noisier convergence
DINOv3	random-init $\rightarrow$ COCO-init student	Similarity jumps 0.42 $\rightarrow$ 0.74 (less discriminative)

B. Four-way SSL comparison at 20% labels

Table VII reports test-set headline metrics for all four SSL-initialised detectors and the random-init / COCO-pretrained baselines, all fine-tuned at the identical 20% labelled budget (Section III-C).

TABLE VII: Four-way SSL comparison and baselines, test set, 20% labelled budget.

Initialisation	Prec.	Rec.	mAP50	mAP50-95
SimCLR (winner)	0.175	0.235	0.136	0.0642
BYOL	0.198	0.238	0.139	0.0631
I-JEPA	0.212	0.207	0.133	0.0601
DINOv3 (distilled)	0.203	0.171	0.102	0.0396
Random-init	0.161	0.208	0.112	0.0503
COCO-pretrained	0.486	0.404	0.416	0.2767

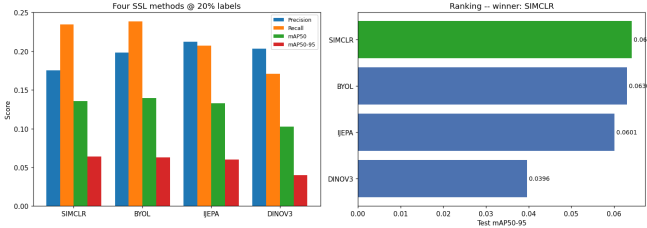


Fig. 13: Four-way SSL comparison at the 20% labelled budget: metric breakdown (left) and mAP50-95 ranking (right).

SimCLR wins (mAP50-95 = 0.0642), narrowly ahead of BYOL (0.0631) and I-JEPA (0.0601), with DINOv3 well behind (0.0396), even behind the random-init baseline (0.0503). We re-evaluated the winning checkpoint for fresh diagnostics (Figures 14–15). We note a baseline-fairness caveat: each downstream notebook independently re-ran its own random-init/COCO baseline rather than sharing one canonical run, producing small (few-percentage-point) discrepancies across notebooks that do not affect the four-way SSL ranking but are reported transparently.

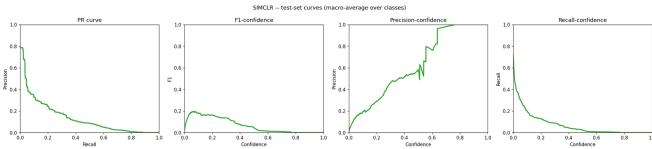


Fig. 14: Test-set PR, F1, precision, and recall vs. confidence curves for the winning SimCLR-initialised detector.

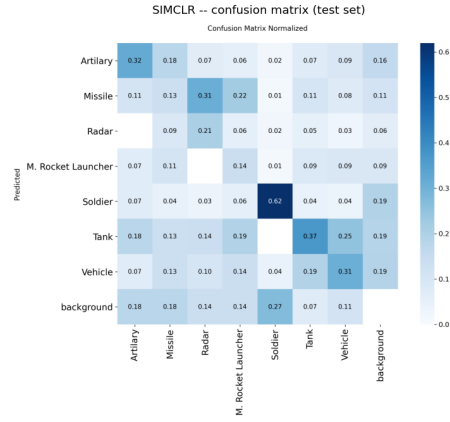


Fig. 15: Normalised confusion matrix for the winning SimCLR-initialised detector.

C. Tracking

The winning SimCLR-initialised detector was deployed over two video clips (a real YouTube clip and an AI-generated clip) using ByteTrack, with a BoT-SORT comparison on one clip. Figure 16 shows representative annotated frames. In the absence of ground-truth track annotation, Figure 17 reports proxy metrics: unique track-ID count versus a manually-established expected object count, and the per-frame detection confidence distribution. Because tracking-by-detection is fundamentally bounded by detector quality (a missed detection breaks a track; a false positive can seed an incorrect one), and the underlying detector’s absolute mAP50-95 is modest (0.0642, Table VII), we observed track fragmentation consistent with this ceiling, a direct illustration of how Part B’s detector-quality finding propagates downstream into tracking quality (elaborated in Section VI).

D. Bonus: label-efficiency ablation

We extended the single-point (20%) comparison to a five-point nested grid ($\rho \in \{0.10, 0.20, 0.30, 0.40, 0.50\}$) for SimCLR-init versus COCO-init, with $D_{0.1} \subset D_{0.2} \subset \dots \subset D_{0.5}$ constructed around the existing $\rho = 0.20$ subset to guarantee nesting and permit reuse of the $\rho = 0.20$ results without retraining. Table VIII and Figure 18 report the resulting curve, reusing a dashed reference line at the Part A YOLOv26-s fully-supervised (100%-labelled) test mAP50-95 of 0.460.

TABLE VIII: Label-efficiency grid: test mAP50-95 by label fraction.

Label fraction	SimCLR-init	COCO-init
10%	0.0280	0.1680
20% (reused)	0.0642	0.2767
30%	0.0967	0.3349
40%	0.0898	0.3788
50%	0.1431	0.3761

The COCO-initialised detector dominates SimCLR-init at every tested fraction; even SimCLR-init at 50% (0.1431) trails



Fig. 16: Representative annotated frames from ByteTrack applied to both video clips (real YouTube footage, top; AI-generated footage, bottom), showing detection boxes, persistent track IDs, and confidence scores.

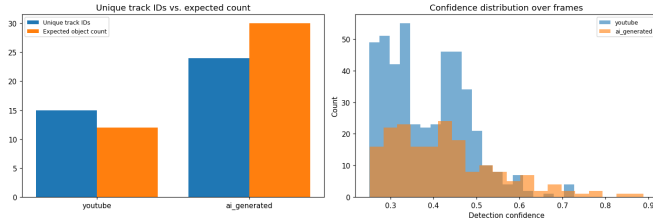


Fig. 17: Proxy tracking metrics: unique track-ID count vs. expected object count, and confidence distribution over frames, for both clips.

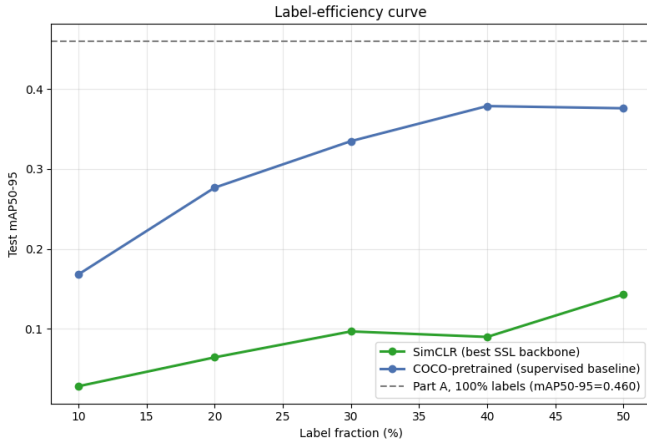


Fig. 18: Label-efficiency curve: test mAP50-95 vs. label fraction, SimCLR-init vs. COCO-init, with the Part A fully-supervised reference (dashed).

COCO-init at its worst tested point, 10% (0.1680). Break-even, defined as the smallest ρ at which SimCLR-init matches COCO-init@50% or 95% of the Part A reference, is not reached within $\rho \in [0.10, 0.50]$. Two small non-monotonic dips (SimCLR: 30%→40%; COCO: 40%→50%) are consistent with expected training variance at a 50-epoch, patience-15 budget on small subsets, rather than a methodological error. Figure 19 shows qualitative predictions at $\rho = 0.10$ vs. $\rho = 0.50$.

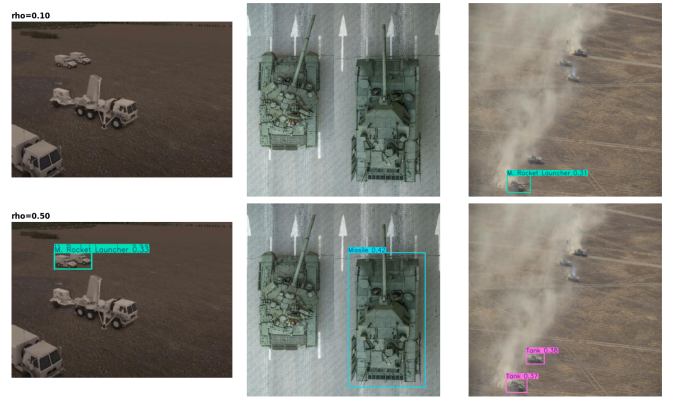


Fig. 19: SimCLR-initialised detector predictions at $\rho = 0.10$ (top) vs. $\rho = 0.50$ (bottom) on three fixed test images.

VI. DISCUSSION AND INSIGHTS

Why did SimCLR win, mechanistically? SimCLR’s explicit negative-pair contrastive objective provides a direct discriminative signal that converges efficiently even on a modest-sized (1,339-image), visually homogeneous pool of all top-down military aerial imagery, where instance discrimination between images is a comparatively tractable pretext task (Figure 12 shows visibly separable class clusters, supporting this). BYOL’s EMA-based, negative-free objective is known in the SSL literature to benefit most from very large, visually diverse corpora; our narrow, single-domain pool does not provide this diversity, which is consistent with (though does not conclusively prove) BYOL’s narrowly weaker result. As an alternative explanation, we considered whether BYOL’s small deficit instead reflects residual instability from the collapse episodes encountered during hyperparameter search (Section III-B). We consider this plausible but secondary, since the reported run’s loss curve (Figure 9) shows stable, converged training with no visible collapse signature at the epoch range used for the transferred checkpoint.

Where does SSL help most, and does our data match theory? Standard label-efficiency theory predicts the SSL/supervised gap should be *largest at low label fractions* (where a pretrained head-start matters most) and should *narrow* as labelled data grows. Our label-efficiency ablation

(Figure 18) shows the opposite: the gap remains wide and non-narrowing across the entire 10–50% range. This is the report’s central, honestly-reported negative finding: SSL pretraining confined to a small (1,339-image), narrow-domain, single-dataset pool does not substitute for large-scale supervised (COCO) pretraining in this problem setting, at any label budget we tested.

Why did DINOv3 under-perform despite its more powerful teacher? We attribute this to two gaps that had to be bridged simultaneously within the shared 200-epoch budget: a domain gap (DINOv3’s web-scale teacher distribution vs. our narrow aerial-military domain) and an architecture gap (ViT teacher, CNN student). The other three methods face no analogous gap, since they train a YOLOv26n encoder end-to-end on our own pool without an external teacher’s domain mismatch to overcome. We regard this as evidence that distillation budget must scale with domain gap, not merely with epoch count matched to competing methods. This is directly supported by DINOv3’s loss curve (Figure 11), which is still trending downward at epoch 200, suggesting the distillation had not yet converged within budget.

How did detector quality propagate into tracking? The tracking pipeline (Section V) inherits the SimCLR-initialised detector’s modest absolute mAP50-95 (0.0642). Because tracking-by-detection has no independent object-recognition capability of its own, every missed or spurious per-frame detection directly threatens track continuity; the proxy metrics (Figure 17) are consistent with this propagation, reinforcing that Part B’s central finding (weak SSL backbones on this domain) is not confined to the detection metric table but visibly affects every downstream consumer of the detector, including tracking.

Practical annotation-budget recommendation. For a team facing a similarly narrow-domain, small-scale detection problem and choosing between investing effort in an SSL pretraining pipeline versus simply annotating more images with a COCO-transfer baseline, our results favour the latter without qualification within the range we tested: every additional labelled percentage point produced a larger downstream mAP50-95 gain under COCO initialisation than switching from COCO to SimCLR initialisation ever could. We report this as an actionable, domain-specific recommendation rather than a general claim about SSL’s value.

VII. LIMITATIONS AND FUTURE WORK

Compute-constrained pretraining. All four SSL methods were pretrained for a fixed 200 epochs on two T4 GPUs; a substantially longer budget, particularly for DINOv3’s distillation (whose loss curve had not yet plateaued, Figure 11), might change its relative standing.

Single dataset, single domain. All findings are specific to KIIT-MiTA’s narrow, top-down military aerial domain and its particular scale (1,681 images); we make no claim that SSL pretraining is generally weak, only that it under-performed supervised pretraining *in this setting*.

Synthetic-video caveat. One of our two tracking clips is AI-generated; synthetic footage can differ systematically from real footage in texture, motion, and lighting, so tracking results on that clip are reported alongside, not in place of, the real-clip’s results.

What a larger study would add. A 1%-label extension of the label-efficiency ablation would test whether SSL’s theoretical advantage emerges at the most extreme low-label regime even though it did not emerge across 10–50%; a larger unlabelled pool (beyond our 1,339-image SSL pool) would test whether our central negative finding is a function of pool size specifically, or of domain narrowness more fundamentally. These remain open questions this report’s compute budget could not resolve.

VIII. CONCLUSION

Among four SSL pretraining methods SimCLR is the strongest (mAP50-95 = 0.0642 at 20% labels), but no SSL method, including SimCLR at any tested label fraction up to 50%, approaches the COCO-pretrained supervised baseline (0.2767 at 20%; 0.3761 at 50%), and this gap does not narrow as theory predicts. We conclude that for narrow-domain, small-scale detection problems structurally similar to military drone imagery, annotation investment is a more reliable path to downstream performance than small-scale SSL pretraining engineering, and we deploy and report our best available SSL-initialised detector’s tracking performance honestly, including its detector-quality-limited proxy metrics, rather than substituting a stronger but non-SSL checkpoint.

REFERENCES

- [1] KIIT-MiTA Dataset, Mendeley Data. [Online]. Available: <https://data.mendeley.com/datasets/drjmr5k5/1>
- [2] M. R. A. Rashid, “ssl-detection-lab,” 2024. [Online]. Available: <https://github.com/rifat963/ssl-detection-lab>
- [3] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLO,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [4] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, “YOLOv10: Real-time end-to-end object detection,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [5] Y. Tian, Q. Ye, and D. Doermann, “YOLOv12: Attention-centric real-time object detectors,” *arXiv preprint arXiv:2502.12524*, 2025.
- [6] Roboflow, “RF-DETR: A real-time, DETR-based object detection model,” 2025. [Online]. Available: <https://github.com/roboflow/rf-detr>
- [7] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in *Proc. ICML*, 2020, pp. 1597–1607.
- [8] J.-B. Grill, F. Strub, F. Altché, C. Tallec, P. H. Richemond, E. Buchatskaya, C. Doersch, B. A. Pires, Z. D. Guo, M. G. Azar, B. Piot, K. Kavukcuoglu, R. Munos, and M. Valko, “Bootstrap your own latent,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 21271–21284.
- [9] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas, “Self-supervised learning from images with a joint-embedding predictive architecture,” in *Proc. CVPR*, 2023, pp. 15619–15629.
- [10] O. Siméoni *et al.*, “DINOv3,” *Meta AI Research*, 2024.
- [11] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “ByteTrack: Multi-object tracking by associating every detection box,” in *Proc. ECCV*, 2022, pp. 1–21.
- [12] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, “BoT-SORT: Robust associations multi-pedestrian tracking,” *arXiv preprint arXiv:2206.14651*, 2022.

IX. CONTRIBUTION

This statement lists which group member led which notebook(s) and report section(s).

Member	Notebooks / Report Sections Led
Md. Ariful Islam Opi (2023-1-60-141)	<i>EDA Notebook; Notebook 1a/1b – SimCLR and Notebook 5 – Tracking; Part A Notebook – RF-DETR; Report Sections III-B1, V-C</i>
MD Shafin Ahamed Siam (2023-1-60-007)	<i>Notebook 2a/2b – BYOL; Part A Notebook – YOLOv10; Report Section III-B2</i>
Albin Israt Siddique (2023-1-60-212)	<i>Notebook 3a/3b – I-JEPA; Part A Notebook – YOLOv12; Report Section III-B3</i>
Nasrullah Kaisher Sijan (2023-1-60-204)	<i>Notebook 4a/4b – DINOv3; Bonus Notebook; Part A Notebook – YOLOv26; Report Section III-B4</i>